Aris — Analytics Metrics: How Every Number Is Calculated

This document covers every metric shown in the brand dashboard and single-event analytics pages, with the exact formulas and the code that produces them.

All server-side calculation lives in `server/src/services/analyticsService.ts`. Frontend rendering lives in `client/app/brand/dashboard/page.tsx` (brand dashboard) and `client/app/brand/events/[id]/page.tsx` (single event detail).

Table of Contents

1. Raw Counters (simple increments)
2. Live / Current Viewers
3. Engagement Time Estimate
4. Vote Distribution & Content Metrics
5. Entropy
6. Normalized Entropy
7. Winning Margin
8. Historical Alignment
9. Top Content Vote Percent
10. Vote Completion Percent (Voter Capacity)
11. Trust Score
12. Decision Confidence Score (DCS)
13. Demographics (Age & Gender)
14. Time-Series: Views & Votes Over Time
15. Brand Aggregate Averages
16. Clicks Breakdown
17. Follower Growth
18. Dashboard Stats Row (Banner)
19. Insights Tab: Narrative Text
20. Data Flow Summary

---

1. Raw Counters

These are stored in the `EventAnalytics` Prisma model and incremented by dedicated tracking functions. They never compute — they just accumulate.

Counter	Incremented by	Notes
`totalViews`	`trackEventView()`	Every page visit (logged-in or anonymous)
`uniqueParticipants`	`trackEventView()`	Only logged-in users, only on first-ever view
`totalVotes`	`trackVote()`	Called after each vote is cast
`totalSubmissions`	`trackSubmission()`	Called after each submission is created
`totalShares`	`trackShare()`	Called when share button is triggered

`totalClicks`	`trackClick()`	Called on any tracked click target

Unique participant check (`trackEventView`):

```
SELECT ActivityLog WHERE eventId = X AND userId = Y AND type = 'EVENT_VIEW'
→ If no prior log exists → isFirstView = true → increment uniqueParticipants
```

---

2. Live / Current Viewers

What it shows: The number of users actively looking at an event page right now.

Implementation: In-memory presence service (`server/src/services/presenceService.ts`).

```
eventPresence: Map<eventId, Set<userId>>
```

- User opens event page via Socket.IO `join-event` event → `PresenceService.addUser(eventId, userId)` → user's ID added to the Set
- User disconnects or leaves → `PresenceService.removeUser(eventId, userId)` → ID removed
- `getActiveCount(eventId)` → returns `Set.size`

```
// analyticsService.ts — getEventAnalytics()
const currentViewers = getActiveCount(eventId);
```

Socket broadcasts a `presence-update` event to the entire `event:{eventId}` room whenever someone joins or leaves, so the displayed count updates in real-time on the frontend.

Important: This is server-process-local. If you run multiple server instances, counts will not be shared (would need Redis upgrade).

---

3. Engagement Time Estimate

What it shows: Estimated time users spend per event. Used in the "Avg Engagement Time" card in the Stats tab.

This is a frontend-side heuristic — no real time tracking:

```
// dashboard/page.tsx — InsightsTab
const secs = votes * 15 + posts * 45;
```

- Each vote action is estimated at **15 seconds** of engagement
- Each post (submission) action is estimated at **45 seconds** of engagement
- Average across all events: `sum(secs) / eventCount`
- Displayed as `Xm Ys` if ≥ 60 seconds, else just `Xs`

This is a proxy metric — it is not measured from actual time-on-page data.

---

4. Vote Distribution & Content Metrics

Computed per content item (submission or proposal) inside `getDetailedEventAnalytics()` :

```
votePercentage = (item.voteCount / totalVotes) * 100
```

Each content item gets:

Field	Formula
`voteCount`	Raw count from DB (`submission.voteCount` or `proposal.voteCount`)
`votePercentage`	`(voteCount / totalVotes) × 100`
`rank`	`finalRank` stored on submission/proposal
`demographicBreakdown`	Per-gender and per-age-group vote counts (see §13)

Content items are sorted by `voteCount DESC` before metrics are applied.

---

5. Entropy

What it measures: How spread out the votes are across all content options. High entropy = votes evenly distributed (no clear winner). Low entropy = concentrated on one option (strong preference).

Formula — Shannon entropy (base-2):

```
H = -Σ (p_i × log₂(p_i))   for all content items i where voteCount > 0

where p_i = voteCount_i / totalVotes
```

```
// analyticsService.ts
function computeEntropy(voteCounts: number[], totalVotes: number): number {
    if (totalVotes === 0) return 0;
    let entropy = 0;
    for (const count of voteCounts) {
        if (count === 0) continue;
        const p = count / totalVotes;
        entropy -= p * Math.log2(p);
    }
    return entropy;
}
```

Range: 0 to $\log_2(N)$ where N is the number of content options.

- `0` = all votes on one item (maximum clarity)
- `log₂(N)` = perfectly even distribution (maximum confusion)

Displayed on:

- Single event detail → "Analytics & Results" tab → **Entropy** KPI tile (yellow)
- Brand dashboard → Stats tab → "Decision Quality per Event" line chart (dashed amber line, plotted as normalized × 100)
- Insights tab → narrative text

---

6. Normalized Entropy

What it measures: Entropy scaled to [0, 1] regardless of how many content options exist. Makes entropy comparable across events with different numbers of options.

Formula:

```
normalizedEntropy = entropy / log₂(N)
```

```
const normalizedEntropy = numEntries > 1 ? entropy / Math.log2(numEntries) : 0;
```

- `N` = number of content items (submissions or proposals)
- If N ≤ 1, normalizedEntropy = 0 (no comparison possible)
- `0.0` = perfectly concentrated votes (one clear winner)
- `1.0` = perfectly even distribution (no preference signal)

Interpretation shown in single event page:

```
normalizedEntropy < 0.5 → "a clear winner with focused audience preference"
normalizedEntropy ≥ 0.5 → "a spread vote suggesting mixed audience preference"
```

Used in: DCS calculation (see §12), "Decision Quality" line chart

---

7. Winning Margin

What it measures: How far ahead the top-voted content is from second place, as a percentage of total votes.

Formula:

```
winningMargin = ((rank1Votes - rank2Votes) / totalVotes) × 100
```

```
const sorted = [...voteCounts].sort((a, b) => b - a);
const rank1Votes = sorted[0] || 0;
const rank2Votes = sorted[1] || 0;
const winningMargin = totalVotes > 0 ? ((rank1Votes - rank2Votes) / totalVotes) * 100 : 0;
```

- `rank1Votes` = highest vote count among all content items
- `rank2Votes` = second highest vote count
- If there's only one item or no votes → 0

Range: 0% to 100%

- `0%` = first and second place tied
- `100%` = all votes went to one item, nothing to second place

Displayed on:

- Single event detail → **Winning Margin** KPI tile (green)
- Brand dashboard → Stats tab → "Decision Quality per Event" line chart (solid emerald line)
- Insights tab → per-event card "Margin" column

- Insights tab → "Reason" narrative text: `> 20%` → "strong audience preference clarity", else "moderate audience split"

---

8. Historical Alignment

What it measures: The fraction of all votes that went to the top-ranked content item. Essentially the top item's vote share expressed as a ratio.

Formula:

```
historicalAlignment = rank1Votes / totalVotes
```

```
const historicalAlignment = totalVotes > 0 ? rank1Votes / totalVotes : 0;
```

Range: 0.0 to 1.0 (displayed as percentage, e.g. `0.65` → `65%`)

Interpretation: Tells you how aligned your audience was with the eventual winner. High value = most people voted for the same thing. A value of 1.0 means all votes went to one item.

Displayed on:

- Single event detail → Outcome Insights text: `"Historical alignment stands at X%"`
- Brand dashboard → Stats tab → "Decision Quality per Event" line chart (solid violet line)
- Brand dashboard → Stats tab → "Decision Confidence & Quality" panel → **Avg Alignment** metric
- Dashboard Insights tab → "Reason" narrative

---

9. Top Content Vote Percent

What it measures: Same as historical alignment but expressed as a percentage.

```
topContentVotePercent = (rank1Votes / totalVotes) × 100
```

```
const topContentVotePercent = totalVotes > 0 ? (rank1Votes / totalVotes) * 100 : 0;
```

Used in the Insights tab as the per-event DCS proxy:

```
// EventInsightCard
const dcs = summary ? (summary.topContentVotePercent / 100) : 0;
```

The per-event insight card uses `topContentVotePercent / 100` as its local "decision confidence" to drive the next-action recommendation shown in the expanded row.

---

10. Vote Completion Percent (Voter Capacity)

What it measures: What percentage of the event's capacity has been filled by voters.

Formula:

```
voteCompletionPct = min(100, (totalVotes / capacity) × 100)
```

```
const voteCompletionPct = event.capacity
    ? Math.min(100, (totalVotes / event.capacity) * 100)
    : 0;
```

- Uses `event.capacity` — the maximum number of participants the brand set
- Capped at 100% even if votes somehow exceed capacity
- Returns 0 if no capacity is defined

Displayed on:

- Single event detail → **Voter Capacity** KPI tile (blue)

---

11. Trust Score

What it measures: The average trustworthiness of the participants who voted in an event. Each user has a `trustScore` (0.0–1.0) updated via EMA (exponential moving average) after events complete.

Calculation for an event:

```
const voterUserIds = [...new Set(votes.map(v => v.userId))];
const voterUsers = await prisma.user.findMany({
    where: { id: { in: voterUserIds } },
    select: { trustScore: true },
});
const avgParticipantTrustScore = totalTrust / voterUsers.length;
// Default: 0.5 if no voters
```

Calculation for a brand (across all events):

```
const participantUsers = await prisma.user.findMany({
    where: { id: { in: [...uniqueUserIds] } },
    select: { trustScore: true },
});
avgParticipantTrustScore = totalTrust / participantUsers.length;
```

Range: 0.0 to 1.0 (displayed as %, e.g. `0.73` → `73%`)

Used in: DCS formula (see §12) **Displayed on:**

- Brand dashboard → Stats tab → "Decision Confidence & Quality" panel → **Trust Score** row

---

12. Decision Confidence Score (DCS)

What it measures: How reliable and clear the voting outcome was. Combines vote concentration (via normalized entropy) with the trustworthiness of the participants.

Formula:

```
DCS = (1 - normalizedEntropy) × 0.6 + avgParticipantTrustScore × 0.4
```

```
// analyticsService.ts — getBrandAnalytics()
const decisionConfidenceScore = (1 - avgNormalizedEntropy) * 0.6 + avgParticipantTrustScore
* 0.4;
```

Component weights:

- **60%** — vote concentration: `(1 - normalizedEntropy)` → high when votes are focused, low when spread
- **40%** — participant trust: average trust score of all unique voters

Range: 0.0 to 1.0

Interpretation thresholds (DCSGaugeSmall):

Score	Label	Color	Meaning
≥ 0.70	High	Green	Clear signal — proceed confidently
0.45–0.69	Moderate	Amber	Some signal — run additional tests
< 0.45	Low	Red	Weak signal — redesign creative options

Single-event DCS (on the event detail page) is recalculated inline on the frontend:

```
// events/[id]/page.tsx
const dcs = ((1 - eventSummary.normalizedEntropy) * 0.6 +
eventSummary.avgParticipantTrustScore * 0.4) * 100
```

Per-event card DCS (Insights tab) uses a simplified proxy:

```
const dcs = summary.topContentVotePercent / 100
```

Displayed on:

- Brand dashboard → Stats tab → **DCS gauge** (circular SVG progress ring)
- Brand dashboard → Insights tab → "Next Action" recommendation text
- Single event detail → **Decision Confidence** KPI tile (lavender, shown as %)

---

13. Demographics (Age & Gender)

Age Bucketing

Each voter's `dateOfBirth` is converted to an age group server-side:

```
function getAgeGroup(dateOfBirth: Date | null): string {
    const age = Math.floor((now - dateOfBirth) / (365.25 × 24 × 60 × 60 × 1000));
    if (age <= 24)  return '24_under';
    if (age <= 34)  return '25_34';
    if (age <= 44)  return '35_44';
    if (age <= 54)  return '45_54';
    if (age <= 64)  return '55_64';
```

```
    return '65_plus';
}
```

Null `dateOfBirth` → `'unknown'`

Gender Normalization

```
function normalizeGender(gender: string | null): string {
    switch (gender?.toLowerCase().trim()) {
        case 'male':               return 'male';
        case 'female':             return 'female';
        case 'non-binary':         return 'nonBinary';
        case 'prefer not to say':  return 'unknown';
        default:                   return 'other';  // non-null but unrecognized
    }
}
```

Aggregation

For each vote in an event, the voter's `gender` and `dateOfBirth` are fetched and incremented into counters:

```
votesByGender:   { male, female, nonBinary, other, unknown }
votesByAgeGroup: { 24_under, 25_34, 35_44, 45_54, 55_64, 65_plus, unknown }
```

Both overall event totals and per-content-item breakdowns are built.

Frontend — Demographics Chart

The grouped bar chart on the event detail page estimates male/female split within each age group using the overall gender ratio as a multiplier:

```
const malePct   = votesByGender.male   / totalVotes;
const femalePct = votesByGender.female / totalVotes;

demoData = ageKeys.map(k => ({
    age:    labels[k],
    male:   Math.round(votesByAgeGroup[k] * malePct),
    female: Math.round(votesByAgeGroup[k] * femalePct),
}));
```

This assumes the gender ratio is uniform across age groups (an approximation).

---

14. Time-Series: Views & Votes Over Time

Both are computed server-side by bucketing interaction timestamps into **hourly slots**.

Views Over Time

```
const viewInteractions = await prisma.eventInteraction.findMany({
    where: { eventId, type: 'VIEW' },
```

```
    select: { createdAt: true },
    orderBy: { createdAt: 'asc' },
});

// Round each timestamp down to the hour
const hour = new Date(v.createdAt);
hour.setMinutes(0, 0, 0);
hourBuckets.set(key, (hourBuckets.get(key) || 0) + 1);
```

Votes Over Time

Same logic applied to `Vote` records:

```
const votes = await prisma.vote.findMany({
    where: { eventId },
    select: { createdAt: true },
    orderBy: { createdAt: 'asc' },
});
```

Both return `{ timestamp: ISOString, count: number }[]` sorted chronologically.

Frontend merges the two series into a single time map and renders them as an area chart (orange = views, lime = votes) on the single event detail page.

---

15. Brand Aggregate Averages

`getBrandAnalytics()` iterates over all brand events and accumulates metrics only for events that have votes (`eventTotalVotes > 0 && contentVoteCounts.length > 0`).

```
totalHistoricalAlignment += historicalAlignment;
totalEntropy             += entropy;
totalNormalizedEntropy   += normalizedEntropy;
totalWinningMargin       += winningMargin;
completedEventsWithVotes++;

// Then:
avgHistoricalAlignment = totalHistoricalAlignment / completedEventsWithVotes;
avgEntropy             = totalEntropy             / completedEventsWithVotes;
avgNormalizedEntropy   = totalNormalizedEntropy   / completedEventsWithVotes;
avgWinningMargin       = totalWinningMargin       / completedEventsWithVotes;
```

Events with no votes are excluded from these averages so they don't dilute the signal.

The **brand-level DCS** then uses `avgNormalizedEntropy` and the brand-wide `avgParticipantTrustScore` :

```
decisionConfidenceScore = (1 - avgNormalizedEntropy) * 0.6 + avgParticipantTrustScore * 0.4;
```

---

16. Clicks Breakdown

Clicks are stored as `EventInteraction` rows with `type = 'CLICK'` and a `metadata.target` field.

```
// target values and their buckets:
'vote' | 'vote_button' → breakdown.vote
'event'                → breakdown.event
'website'              → breakdown.website
'social'               → breakdown.social
anything else          → breakdown.other
```

The "Clicks Breakdown" donut chart on the brand dashboard Stats tab uses a **frontend approximation** rather than the real breakdown API. It builds the pie data from:

```
pieData = [
    { name: "Vote",    value: totalVotes },
    { name: "Event",   value: totalPosts },
    { name: "Website", value: Math.round(totalViews * 0.35) },   // estimated 35% of views
    { name: "Social",  value: Math.round(totalViews * 0.15) },   // estimated 15% of views
]
```

Website and Social slices are estimates (35% / 15% of total views), not real click data from the API.

---

17. Follower Growth

`getFollowerGrowth()` queries `BrandSubscription` records and buckets `subscribedAt` dates by `day`, `week`, or `month`.

```
// daily key:
key = date.toISOString().split('T')[0]   // "2025-04-01"

// monthly key:
key = `${year}-${month}`                  // "2025-04"

// weekly key: snaps to Monday of that week
```

Returns `{ date, count, delta }[]` where `delta = count - previousBucketCount`.

"Time vs Follower Growth" chart on the Stats tab uses a different source — it plots cumulative `uniqueParticipants` per event as a proxy for audience growth, since follower data requires a separate API call:

```
let cumulative = 0;
analytics.eventsSummary.map(s => {
    cumulative += s.uniqueParticipants;
    return { name, participants: s.uniqueParticipants, cumulative };
})
```

---

18. Dashboard Stats Row (Banner)

The quick-stats shown in the brand banner are computed client-side from the loaded event list:

```
function computeStats(events: Event[]): DashboardStats {
    events.forEach(ev => {
        if (status === "posting" || status === "voting")     liveEvents++;
        if (status === "completed" || status === "cancelled") closedEvents++;
        totalCost  += calculateTotalPool(ev);        // sum of reward pool
        totalViews += ev.eventAnalytics?.totalViews  ?? 0;
        totalVotes += ev.eventAnalytics?.totalVotes  ?? ev._count?.votes ?? 0;
        totalPosts += ev.eventAnalytics?.totalSubmissions ?? ev._count?.submissions ?? 0;
    });
}
```

`calculateTotalPool(ev)` sums `topReward + leaderboardPool + baseReward × participantCount` depending on event type.

---

19. Insights Tab: Narrative Text

The three-panel insight block on the Insights tab generates plain-English text by reading the analytics numbers through threshold logic — there is no LLM involved.

"Result" panel

```
"{totalVotes} total votes across {totalEvents} events with {totalUniqueParticipants} unique
participants.
 Audience is {malePct}% male, {femalePct}% female."
```

"Reason" panel

```
if winningMargin > 20%  → "strong audience preference clarity"
else                    → "moderate audience split"

entropy < 1  → "low decision spread"
entropy 1–2  → "moderate decision spread"
entropy > 2  → "high decision spread"
```

"Next Action" panel (brand level)

```
dcs ≥ 0.75 → "Decision confidence is high (≥75%). Move forward confidently."
dcs ≥ 0.50 → "Decision confidence is moderate (50–75%). Run additional test events."
dcs > 0    → "Decision confidence is low (<50%). Create new events with different ideas."
dcs == 0   → "Run your first events to generate confidence data."
```

Per-event Insights card "Next Action"

Uses `topContentVotePercent / 100` as the local DCS:

```
dcs ≥ 0.75 → "High confidence — move on to the next decision."
dcs ≥ 0.50 → "Moderate confidence — run additional test cases to resolve confusion."
dcs < 0.50 → "Low confidence — create a new event with different ideas."
```

---

20. Data Flow Summary

```
User opens event page
        │
        ▼
Socket.IO 'join-event'
        │
        ├─► PresenceService.addUser()     → currentViewers (in-memory Set)
        │
        └─► AnalyticsService.trackEventView()
                │
                ├─► EventAnalytics.totalViews++
                ├─► EventAnalytics.uniqueParticipants++  (first view only)
                └─► EventInteraction { type: 'VIEW' }     (for timeseries)

User votes
        │
        └─► AnalyticsService.trackVote()
                └─► EventAnalytics.totalVotes++
                    Vote.createdAt                        (for timeseries)
                    Submission.voteCount / Proposal.voteCount (raw scores)

Brand opens dashboard → GET /api/analytics/brand/overview
        │
        └─► getBrandAnalytics(brandId)
                │
                ├─► Per-event:  computeEntropy(), winningMargin, historicalAlignment,
                │                normalizedEntropy, topContentVotePercent, voteCompletionPct
                │
                ├─► Aggregate: avg of all per-event metrics (events with votes only)
                │
                ├─► Trust:     AVG(user.trustScore) across all unique voters
                │
                └─► DCS:       (1 - avgNormalizedEntropy) × 0.6 + avgTrustScore × 0.4

Brand opens single event → GET /api/analytics/events/:id/detailed
        │
        └─► getDetailedEventAnalytics(eventId)
                │
                ├─► Content metrics:  votePercentage per item, demographic breakdowns
                ├─► Decision metrics: entropy, normalizedEntropy, winningMargin,
                │                     historicalAlignment, topContentVotePercent,
voteCompletionPct
                ├─► Trust:            AVG(trustScore) of voters in this event
                ├─► Timeseries:       hourly buckets of VIEW interactions and Vote records
                └─► Demographics:     votesByGender, votesByAgeGroup (per event + per content
item)
```

Key Formulas — Quick Reference

Metric	Formula
Entropy	`H = -Σ (p_i × log₂(p_i))`
Normalized Entropy	`H / log₂(N)`
Winning Margin	`((rank1 - rank2) / totalVotes) × 100`
Historical Alignment	`rank1 / totalVotes`
Top Content Vote %	`(rank1 / totalVotes) × 100`
Vote Completion %	`min(100, totalVotes / capacity × 100)`
DCS (brand-level)	`(1 – normalizedEntropy) × 0.6 + avgTrustScore × 0.4`
DCS (per-event card)	`topContentVotePercent / 100`
Engagement time proxy	`votes × 15s + posts × 45s`
Demo gender split in chart	`ageGroupCount × (genderCount / totalVotes)`